Statistical Code Review Report

1. Introduction

This report presents a statistical analysis of code quality using SonarQube. The objective is to evaluate key metrics such as cyclomatic complexity, method size, and code duplication, and to identify areas for improvement.

---

2. Code Metrics Analysis

2.1 Metrics Collected

The following metrics were analyzed:

- Cyclomatic Complexity
- Lines of Code per Method
- Duplicate Code (%)

2.2 Metrics Summary Table

Module/Class	Avg Cyclomatic Complexity	Max Complexity	Avg Lines per Method	Duplicate Code (%)	Unreachable Code	Notes
config package	Med	High	Med-High	High	Low	SQL duplication
dto package	Low	Low	Low	Med	Low	Field duplicatio
handler package	Med	Med-High	Med	Low-Med	Med	

2.3 Observations

Describe general trends in the metrics:

- Significant duplication of string literals and SQL fragments across config classes.
- Frequent use of generic exception handling across backend services.
- dto layer contains multiple unused or redundant fields, indicating incomplete or evolving design.
- SQL queries are often written using non-optimal patterns (SELECT *).

---

3. Findings and Interpretation

3.1 Key Findings

Identify the most important issues:

- High cyclomatic complexity detected in:
  - AttendanceSQL.java, SessionSQL.java, DatabaseInitializer.java

- Large methods (high LOC per method) found in:
  - SQL query builders in config package
  - Handler Logic in handler package

- Code duplication observed in:
  - SQL column names (student_id, session_id, status)
  - Attendance status strings (present, absent, excused)
  - Repeated SQL fragments

- Unreachable or dead code detected in:
  - Test utility methods
  - Unused variables (rate, teachers, classId)
  - Empty handler/test methods.

3.2 Interpretation of Results

Explain what the metrics mean for code quality:

- High complexity indicates:
  - High complexity indicates code is harder to maintain and increases risk of hidden bugs.

- Large methods suggest:
    - Large methods suggest poor separation of concerns and SRP violations.

- Code duplication leads to:
    - Code duplication increases maintenance cost and inconsistency risk.

- Unreachable code impacts:
    - Unreachable code increases confusion and maintenance overhead.

---

4. Refactoring Recommendations

Suggest improvements based on findings:

- Reduce complexity by:
    - Reduce complexity by splitting SQL logic and introducing service/repository layers.

- Improve method size by:
    - Improve method size by extracting helpers and mappers.

- Eliminate duplication by:
    - Eliminate duplication using constants and enums.

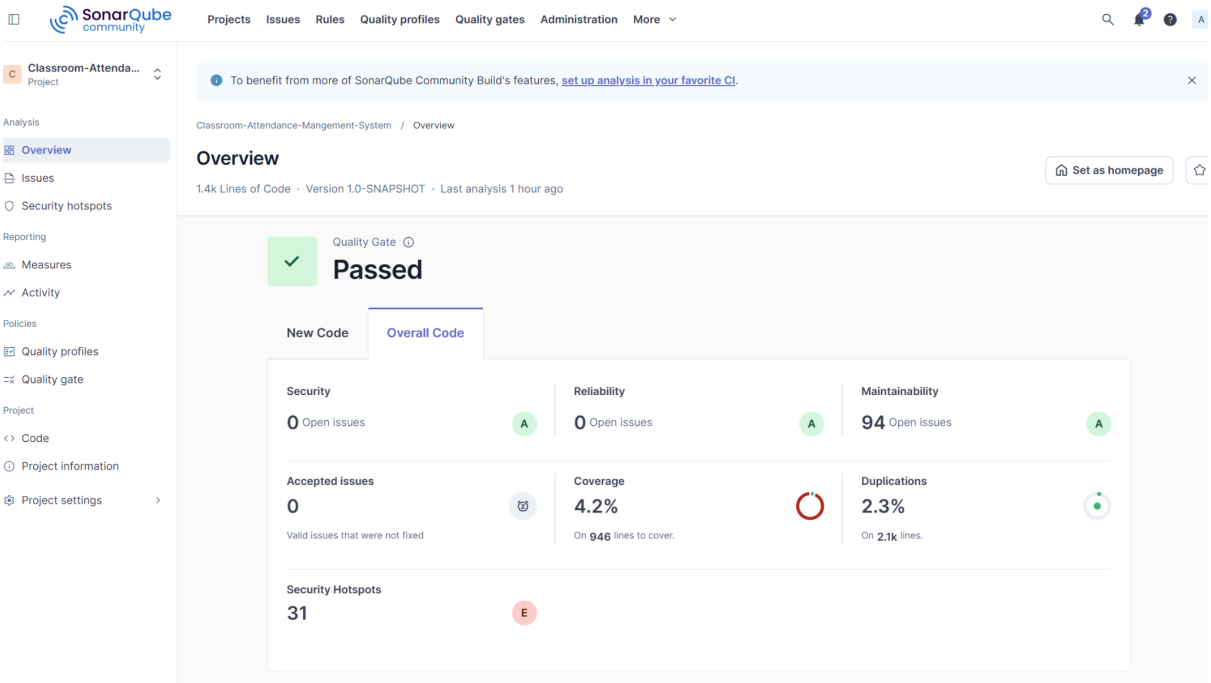
- Remove unreachable code by:
    - Remove unreachable code and unused elements.

---

5. Visual Evidence

5.1 SonarQube Screenshots

- Complexity metrics dashboard
- Code duplication report
- Issues list



---

6. Summary of Metrics

Provide a concise overview:

- Average Cyclomatic Complexity:
- Average Lines per Method:
- Total Duplicate Code:
- Unreachable Code Instances:

---

7. Conclusion

Summarize the overall code quality and improvements needed:

8. Key Improvements

Issue	Impact	Recommended Fix
High Complexity(SQL logic)	Hard to maintain, bug-prone	Break into smaller methods + repositories
Large Methods	Poor readablity	Extract helper methods

Code Duplication	High maintenance cost	Use constants + enums

---

9. Code Review Analysis – Frontend & Integration Focus

9.1 Scope of Contribution

My contribution to the code review focused primarily on:

- Frontend UI modules (Admin, Student, Teacher panels)
- Layout components and shared UI structures
- Navigation logic and page routing
- Code duplication across UI layers
- Maintainability issues in large UI classes

9.2 Key Issues Identified

1. High Code Duplication in UI Layouts

Significant duplication was identified across:

- `AdminAppLayout.java`
- `AppLayout.java`
- `StudentPageSupport.java`
- `TeacherPageSupport.java`

These classes implement very similar sidebar, navigation, and layout logic with minimal variation.

Impact:

- Increases maintenance effort
- Changes must be repeated across multiple files
- Higher risk of inconsistency between roles (Admin/Student/Teacher)

---

2. Repeated Navigation Implementations

Multiple classes define identical `Navigator` implementations:

```
@Override public void goDashboard() { router.go(...); }
@Override public void goTakeAttendance() { router.go(...); }
@Override public void goReports() { router.go(...); }
```

```
@Override public void goEmail() { router.go(...); }
@Override public void logout() { ... }
```

Impact:

- Violates DRY principle
- Harder to update routing behavior globally
- Leads to duplicated logic across modules

---

3. Large UI Classes with Multiple Responsibilities

Classes such as:

- `AdminAttendanceReportsPage`
- `AdminManageClassesPage`
- `StudentAttendancePage`

contain:

- UI construction
- API calls
- Data mapping
- Event handling

all in a single class.

Impact:

- Difficult to test and debug
- Poor separation of concerns
- Reduced readability

---

4. Inconsistent UI Structure Across Pages

Mismatch observed between:

- Student Dashboard
- Student Attendance Page
- Other UI pages

Example:

- Different stat card structures (icon placement, layout inconsistencies)

Impact:

- Poor user experience consistency
- Harder to maintain shared styles
- Increased UI duplication

---

5. Hardcoded UI Values and Repeated Strings

Examples include:

- Icons ("✓", "✕", etc.)
- Status labels ("present", "absent", "excused")
- Inline styling values

Impact:

- Difficult to update UI globally
- Breaks localization consistency
- Increases duplication

9.3 Refactoring Actions Performed

To address the issues identified, the following improvements were implemented:

1. Reduced UI Duplication

- Refactored layout usage through shared helpers (`StudentPageSupport`, etc.)
- Unified sidebar wrapping logic

2. Standardized UI Components

- Aligned stat card design across:
  - Dashboard
  - Attendance pages
- Ensured consistent layout and styling

3. Improved Navigation Structure

- Centralized navigation patterns
- Reduced repeated `Navigator` logic usage where possible

4. Enhanced Code Readability

- Broke down large UI blocks into smaller helper methods
- Improved naming and structure consistency

9.4 Remaining Issues / Future Improvements

Despite improvements, the following areas still require further work:

- Full unification of `AppLayout`, `AdminAppLayout`, and role-specific layouts into a single reusable system
- Creation of shared UI component library (cards, tables, filters)
- Extraction of API logic from UI classes into service layers
- Replacement of hardcoded values with constants or enums
- Further reduction of duplication detected by SonarQube (currently ~8%)

9.5 Contribution Summary

Area	Contribution
UI Refactoring	Reduced duplication and standardized layouts
Navigation	Improved routing consistency
Code Quality	Identified and addressed duplication and large classes
Maintainability	Improved structure and readability
SonarQube Issues	Contributed to reducing duplication and complexity

---

10. Personal Reflection

The code review process highlighted the importance of:

- Avoiding duplication early in development
- Separating UI, logic, and data layers
- Designing reusable components instead of repeating patterns

Applying these improvements resulted in cleaner, more maintainable code and better UI consistency across the application.